```java
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Scanner;

public class Notes {
    public static void main(String[] args) {



        //      == PHASE 1: The Basics & Input ==

        // Setting up the Scanner
        Scanner in = new Scanner(System.in); // Creating scanner object

        // 1D Array Declaration

        // Syntax A: Definition with size (Empty)
        int[] arr = new int[5]; // Creates a box of 5 slots. All are 0 by default.

        // Syntax B: Definition with values (Populated)
        int[] nums = {1, 2, 3, 4}; // Creates size 4 and fills it immediately.

        /*
        FAANG Note: In an interview, if you know data beforehand, always use Syntax B.
        It is cleaner and faster to write.
         */



        //      == PHASE 2: Iteration (Looping) ==

        // 1. The Standard for Loop
        for (int i = 0; i < arr.length; i++) {
            arr[i] = in.nextInt(); // We need 'i' to tell the computer WHERE to put the data
        }

        // 2. The Enhanced for Loop (For-Each)
        for (int num: arr) {
            System.out.print(num + " ");
        }

        System.out.println();

        System.out.println(Arrays.toString(arr));



```

```java
//      == PHASE 4: 2D Arrays (Matrics) ==

// Syntax
int[][] arr2D = new int[3][3]; // 3 rows, 3 columns

// Input Logic
for (int row = 0; row < arr2D.length; row++) {
    for (int col = 0; col < arr2D.length; col++) {
        arr2D[row][col] = in.nextInt();
    }
}


//      == PHASE 5: Jagged Arrays ==

// Syntax
int[][] arrJagged = {
        {1, 2, 4, 4},
        {5, 6,},
        {7, 8, 9}
};

// Dynamic Initialisation: Define rows later
String[][] arrDynamic = new String[3][]; // Define rows only
arrDynamic[0] = new String[4]; // Make row 0 size 4
arrDynamic[1] = new String[2]; // Make row 1 size 2
arrDynamic[3] = new String[3]; // Make row 2 size 3

/*
FAANG note: This is used in optimisation. If you are storing comments on a post, one post might have
1,000 comments and another might have 0. A jagged array saves memory compared to a fixed square matrix.
 */



//      == PHASE 6: ArrayList (Dynamic Arrays) ==

// Syntax: Note: You cannot use primitives (int) here. You must use Wrapper Classes (Integer).
ArrayList<Integer> list = new ArrayList<>(5);
list.add(23); // Adds to the end
list.add(45);

// Continuous Input: A common pattern to read input the input stops
while (in.hasNextInt()) {
    list.add(in.nextInt());
}


```

```java


        //      == PHASE 7: Multidimensional ArrayList ==

        // Syntax
        ArrayList<ArrayList<Integer>> multiAl = new ArrayList<>();

        // Initialisation (Crucial Step): Unlike a 2D array, you cannot just say list.get(0).add(1).
        // You must initialise the inner lists first!
        for (int i = 0; i < 3; i++) { // Create the empty lists inside the main list
            multiAl.add(new ArrayList<>());
        }

        // Now you can add data
        multiAl.get(0).add(10); // Go to list 0, add 10

        /*
        FAANG Note: This Structure is exactly how Adjacency Lists are built. An Adjacency List is the
        primary way we represent Graphs (like connections in LinkedIn or Facebook). You will use this
        structure constantly in advanced interviews.
         */

    }




    //      == PHASE 3: Memory & References (Critical) ==

    static void change (int[] arr) {
        arr[0] = 99; // This modifies the ORIGINAL array in main()!!!!!!!!!!
    }


}

```